



回顾



人工智能与
Python程序设计

- 1. 文件概述
- 2. 文件读写
- 3. 多维数据的格式化与处理
- 4. os模块编程



《人工智能与Python程序设计》— 面向对象编程（一）



人工智能与Python程序设计 教研组



Python 面向对象编程（一）

- 面向对象编程（Object Oriented Programming, OOP）是一种程序设计思想。OOP把对象作为程序的基本单元，一个对象包含了数据和操作数据的函数



Python 面向对象编程（一）

- 面向对象编程（Object Oriented Programming, OOP）是一种程序设计思想。OOP把对象作为程序的基本单元，一个对象包含了数据和操作数据的函数
- 面向过程（Procedure Oriented Programming, POP）的程序设计把计算机程序视为一系列的命令集合，即一组函数的**顺序执行**。为了简化程序设计，面向过程把函数继续切分为子函数，即把大块函数通过切割成小块函数来降低系统的复杂度。



Python 面向对象编程（一）

- 面向对象编程（**Object Oriented Programming, OOP**）是一种程序设计思想。OOP把对象作为程序的基本单元，一个对象包含了数据和操作数据的函数
- 面向过程（**Procedure Oriented Programming, POP**）的程序设计把计算机程序视为一系列的命令集合，即一组函数的**顺序执行**。为了简化程序设计，面向过程把函数继续切分为子函数，即把大块函数通过切割成小块函数来降低系统的复杂度。
- 面向对象的程序设计把计算机程序视为一组对象的集合，而每个对象都可以接收其他对象发过来的消息，并处理这些消息，计算机程序的执行就是一系列消息在各个对象之间传递。



Python 面向对象编程（一）



在Python中，**所有数据类型**都可以视为对象，也可以自定义对象。自定义的对象数据类型就是面向对象编程中的类（class）的概念。



Python 面向对象编程 (一)

- 以 " 处理学生成绩 " 举例, 我们来说明POP 与OOP 在程序流程上的不同之处:

假如有几个学生和他们的考试成绩, 在POP中可用一个dict进行表示:

```
std1 = {'name': 'Michael', 'score': 98 }  
std2 = {'name': 'Bob', 'score': 81 }  
std3 = {'name': 'Kristen', 'score': 93 }
```

如果想处理学生成绩, 可通过函数实现, 如打印学生的成绩:

```
def print_score(std):  
    print('%s: %d' % (std['name'], std['score']))  
  
print_score(std1)  
  
Michael: 98
```



Python 面向对象编程 (一)

- 以 " 处理学生成绩 " 举例，我们来说明POP 与OOP 在程序流程上的不同之处：

若采用OOP，首先考虑的不是程序的执行流程，而是观察这些学生的**共性特点**，定义一个Student 类型，其实例（即Student 对象）拥有name和score这两个**共有属性**（Property）。

```
std1 = { 'name': 'Michael', 'score': 98 }  
std2 = { 'name': 'Bob', 'score': 81 }  
std3 = { 'name': 'Kristen', 'score': 93 }
```




Python 面向对象编程 (一)

若要打印一个学生的成绩，先创建出这个学生对应的对象，然后给对象发送一个打印（print_score）消息，让对象把自己的数据打印出来。

```
class Student(object):  
  
    def __init__(self, name, score):  
        # 类的构造函数, self代表类的实例  
        self.name = name  
        self.score = score  
  
    def print_score(self):  
        # 类的方法  
        print('%s: %d' % (self.name, self.score))
```



Python 面向对象编程 (一)



给对象发消息实际上就是调用对象对应的关联函数，我们称之为对象的方法 (Method)：

```
Michael = Student('Michael', 98) # 创建实例对象
Kristen = Student('Kristen', 93) # 创建实例对象

Michael.print_score() # 调用类的方法
Kristen.print_score() # 调用类的方法
```

```
Michael: 98
```

```
Kristen: 93
```



提纲



人工智能与
Python程序设计

- 1. 类和实例
- 2. 数据封装
- 3. 访问限制



类和实例

- OOP 的设计思想来源于自然界，因为在自然界中，类(class)和实例(instance)的概念非常自然。
 - 类(class): 用来描述具有相同的属性和方法的对象的集合。比如我们定义的Class--Student，是指学生这个概念。
 - 实例(instance): 创建一个类的实例，类的具体对象。比如一个个具体的Student，Michael和Kristen是两个具体的student。



类和实例

- 类和实例是OOP中最为重要的概念
 - 类是抽象的模板，比如Student 类，
 - 实例是根据类创建出来的一个个具体的“对象”，每个对象都拥有相同的方法，但各自的数据可能不同。

```
Michael = Student('Michael', 98) # 创建实例对象  
Kristen = Student('Kristen', 93) # 创建实例对象
```

```
Michael.print_score() # 调用类的方法  
Kristen.print_score() # 调用类的方法
```

```
Michael: 98  
Kristen: 93
```



类和实例

- 仍以“处理学生成绩”举例，我们来说明类和实例的概念
 - Python中，定义类是通过`class`关键字
 - `class` 后为类名，类名通常是大写开头的单词
 - 类名 (`object`)，表示该类从`object`父类继承下来。通常，如果没有合适的继承类，就使用`object`类，这是所有类最终都会继承的类。

```
class Student2(object):  
    pass
```




类和实例

- 仍以“处理学生成绩”举例，我们来说明类和实例的概念
 - 定义好了Student2 类，就可以根据它来创建出其实例

```
Michael = Student2()  
Kristen = Student2()
```

```
print(Michael)  
print(Kristen)
```

```
<__main__.Student2 object at 0x7fad4b7d5ca0>  
<__main__.Student2 object at 0x7fad4b7d5730>
```

```
print(Student2)
```

```
<class '__main__.Student2'>
```



类和实例

- 仍以“处理学生成绩”举例，我们来说明类和实例的概念
 - 定义好了Student2 类，就可以根据它来创建出其实例

```
Michael = Student2()  
Kristen = Student2()
```

```
print(Michael)  
print(Kristen)
```

```
<__main__.Student2 object at 0x7fad4b7d5ca0>  
<__main__.Student2 object at 0x7fad4b7d5730>
```

```
print(Student2)
```

```
<class '__main__.Student2'>
```

Student2 本身是一个类，而Michael指向的是一个Student2 的实例，0x7fef7387a160是其内存地址，每个object 的地址都不一样。而Student2本身则是一个类



- struct rec {
 int x;
 int y;
• }



类和实例

- 仍以“处理学生成绩”举例，我们来说明类和实例的概念
 - Student2类实例化后，可以自由地给一个实例变量绑定属性

```
Michael.name="Michael Simon"  
print(Michael.name)
```

```
Michael Simon
```

```
Kristen.name
```

```
-----  
-----  
AttributeError                                Traceback (most recent call  
1 last)  
<ipython-input-24-41b55b5ff35e> in <module>  
----> 1 Kristen.name  
  
AttributeError: 'Student' object has no attribute 'name'
```



类和实例

- 类起到模板的作用，可以在创建实例的时候，把一些我们认为必须绑定的属性强制填写进去。
- 通过定义一个特殊的 `__init__` 方法（构造函数），在创建实例的时候，就把 `name`，`score` 等属性绑上去
 - `__init__` 前后分别有两根下划线
 - `__init__()` 的第一个参数永远是 `self`，表示创建的实例本身。因此，在 `__init__()` 内部，就可把各种属性绑定到 `self`，因为 `self` 就指向创建的实例本身。

```
class Student(object):  
  
    def __init__(self, name, score):  
        # 类的构造函数, self代表类的实例  
        self.name = name  
        self.score = score
```



类和实例

- 有了 `__init__()`，在创建实例时，就不能传入空的参数了，必须传入与 `__init__()` 相匹配的参数，但 `self` 不需要传，Python 解释器自己会把实例变量传进去。

```
Michael = Student('Michael', 98) # 创建实例对象  
print(Michael.name)  
print(Michael.score)
```

```
Michael  
98
```




类和实例

- 有了 `__init__()`，在创建实例时，就不能传入空的参数了，必须传入与 `__init__()` 相匹配的参数，但 `self` 不需要传，Python 解释器自己会把实例变量传进去。

```
Michael = Student('Michael', 98) # 创建实例对象
print(Michael.name)
print(Michael.score)
```

```
Michael
98
```

和普通的函数相比，在类中定义的函数只有一点不同，就是第一个参数永远是实例变量 `self`，并且调用时，**不用传递该参数**。除此之外，类的方法和普通函数没有显著区别，所以，仍然可以用默认参数、可变参数、关键字参数和命名关键字参数。



提纲



人工智能与
Python程序设计

- 1. 类和实例
- 2. 数据封装
- 3. 访问限制



数据封装

- OOP 的一个重要特点就是**数据封装**。
 - Student类中，每个实例拥有各自的name和score数据
 - 可以通过函数来访问实例的数据，如打印某个学生的成绩

```
def print_score(std):  
    print('%s: %d' % (std.name, std.score))  
  
print_score(Michael)
```

Michael: 98



数据封装

- Student 的实例本身就拥有这些数据，故要访问它们，可以直接在 Student 类的内部定义访问数据的函数，进而把“数据”封装。
- 封装数据的函数是和Student类本身是关联起来的，我们称之为**类的方法**。



数据封装

- Student 的实例本身就拥有这些数据，故要访问它们，可以直接在 Student 类的内部定义访问数据的函数，进而把“数据”封装。
- 封装数据的函数是和Student类本身是关联起来的，我们称之为**类的方法**。

```
class Student(object):  
  
    def __init__(self, name, score):  
        # 类的构造函数, self代表类的实例  
        self.name = name  
        self.score = score  
  
    def print_score(self):  
        # 类的方法  
        print('%s: %d' % (self.name, self.score))
```



数据封装

- Student 的实例本身就拥有这些数据，故要访问它们，可以直接在 Student 类的内部定义访问数据的函数，进而把“数据”封装。
- 封装数据的函数是和Student类本身是关联起来的，我们称之为**类的方法**。

```
class Student(object):  
  
    def __init__(self, name, score):  
        # 类的构造函数, self代表类的实例  
        self.name = name  
        self.score = score  
  
    def print_score(self):  
        # 类的方法  
        print('%s: %d' % (self.name, self.score))
```

我们发现，要定义一个方法，除了第一个参数是self外，其他和普通函数一样。



- Class `str(xxx)`:
 - `def replace(self, src, tgt):`
 - `def join(self, [])`



数据封装

- 要调用一个方法，只需要在实例变量上直接调用即可。除了self不用传递，其他参数正常传入。
 - 从调用方来看Student 类，只需在创建实例时给定name和score
 - score打印会在Student 类的内部实现。这些数据和逻辑被『封装』起来，调用会变得容易。

```
Michael = Student('Michael Simon', 98)
Michael.print_score()
```

```
Michael Simon: 98
```

数据封装

- 通过封装，可以给class增加新的方法。

```
class Student(object):  
  
    def __init__(self, name, score):  
        # 类的构造函数, self代表类的实例  
        self.name = name  
        self.score = score  
  
    def print_score(self):  
        # 类的方法  
        print('%s: %d' % (self.name, self.score))  
  
    def get_grade(self):  
        if self.score >= 90:  
            return 'A'  
        elif self.score >= 60:  
            return 'B'  
        else:  
            return 'C'
```




提纲



人工智能与
Python程序设计

- 1. 类和实例
- 2. 数据封装
- 3. 访问限制



访问限制

- 在class内部，可以有属性和方法，而外部代码可以通过直接调用实例变量的方法来操作数据，从而隐藏了内部的复杂逻辑。
 - 从示例Student类的定义来看，外部代码可以修改一个实例的属性

```
Michael = Student('Michael Simon', 98)
Michael.print_score()
Michael.score=80
Michael.print_score()
```

```
Michael Simon: 98
```

```
Michael Simon: 80
```



访问限制

- 在class内部，可以有属性和方法，而外部代码可以通过直接调用实例变量的方法来操作数据，从而隐藏了内部的复杂逻辑。
 - 从示例Student类的定义来看，外部代码可以修改一个实例的属性
 - 为了让内部属性不被外部访问，可以把属性的名称前加上两个下划线__，使其变为一个私有变量(private)，未加下划线则为公有变量(public)
 - 私有变量只有内部可以访问，外部不能访问；
 - 公有变量内部、外部皆可以访问

访问限制



```
class Student(object):  
  
    def __init__(self, name, score):  
        self.__name = name # 私有变量  
        self.__score = score # 私有变量  
  
    def print_score(self):  
        print('%s: %d' % (self.__name, self.__score))
```

```
Michael = Student('Michael', 98) # 创建实例对象  
Michael.print_score()  
print(Michael.__score)
```

Michael: 98

```
-----  
AttributeError                                Traceback (most recent call last)  
<ipython-input-26-bb01d8fad8b6> in <module>  
      1 Michael = Student('Michael', 98) # 创建实例对象  
      2 Michael.print_score()  
----> 3 print(Michael.__score)  
  
AttributeError: 'Student' object has no attribute '__score'
```

访问限制



```
class Student(object):  
  
    def __init__(self, name, score):  
        self.__name = name  
        self.__score = score  
  
    def print_score(self):  
        print('%s: %s' % (self.__name, self.__score))
```

```
Michael = Student('Michael', 98) # 创建实例对象  
Michael.print_score()  
Michael.__score = 80  
Michael.print_score()
```

Michael: 98

Michael: 98



访问限制

- 通过引入私有变量，可以确保外部代码不能随意修改对象内部的状态。即，通过访问限制的保护，使得代码更加健壮。
- 如果外部代码想访问、修改私有变量，可通过增加类的方法进行实现。

```
class Student(object):  
    ...  
  
    def get_name(self):  
        return self.__name  
  
    def get_score(self):  
        return self.__score  
  
    def set_score(self, score):  
        self.__score = score
```


访问限制



那么，类的方法会比直接通过外部访问/修改有什么优势呢？

```
class Student(object):  
    ...  
  
    def set_score(self, score):  
        if 0 <= score <= 100:  
            self.__score = score  
        else:  
            raise ValueError('bad score')
```

在类的方法中，可以对参数做相关的检查，避免出现异常错误。



访问限制

- 注意：
 - 类似于_`xxx`的变量是`private`变量，外部代码不能直接访问。
 - 类似于_`xxx`的变量允许外部代码访问，但按照约定俗成的规定，当你看到这样的变量时，意思就是，“虽然我可以被访问，但请把我视为`private` 变量，不要随意访问”
 - 类似于_`xxx_`的变量是特殊变量，不是`private` 变量，外部代码可直接访问。



- Class rec{
 - Private:
 - Int x;
 - Int y;
- }



访问限制

- 再回来分析这个示例：

```
Michael = Student('Michael Simon', 98)
Michael.print_score()
Michael.__score=80
Michael.print_score()
```

```
Michael Simon: 98
Michael Simon: 98
```

```
print(Michael._Student__score)
print(Michael.__score)
```

```
98
80
```

内部的__score变量已经被Python解释器自动改成了_Student__score，外部代码给Michael实例新增了一个__score变量



回顾



人工智能与
Python程序设计

- 1. 类和实例
- 2. 数据封装
- 3. 访问限制



谢谢！